

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
SCHOOL OF INFORMATION COMMUNICATION TECHNOLOGY



SOICT

Capstone Project Report:

SMS Spam Classifier

Group members:

Pham Thi Ngoc Anh - 20235473

Tran Mai Duong - 20235493

Le Quang Huy - 20235503

Vu Thuy Linh - 20235521

Supervised by:

Assoc. Prof. Nguyen Hong Thanh & PhD. Dinh Viet Sang

Hanoi, 2025

Abstract

This project presents the design and implementation of an SMS spam classification system using both machine learning and deep learning approaches. Various models including K-Nearest Neighbors, SVM, Naive Bayes, Logistic Regression and LSTM were trained and evaluated on a dataset collected from multiple sources. Standard preprocessing techniques and feature extraction methods such as Bag-of-Words and TF-IDF were applied. Experimental results show that ensemble learning and deep neural networks achieve high accuracy and strong generalization, making them effective for spam detection.

Through this project, we gained practical experience in building AI-powered systems for real-world applications in cybersecurity. We would like to express our sincere thanks to Ph.D. Dinh Viet Sang and Assoc. Prof. Nguyen Hong Thanh for your guidance and support throughout this course.

Contents

1	Introduction	1
1.1	Project description	1
1.2	Work distributions	1
1.3	The Dataset and Exploratory Data Analysis	1
2	Preprocessing Data	2
2.1	Cleaning and tokenizing texts	2
2.2	Vectorizing texts	3
3	Machine Learning algorithms	4
3.1	K-Nearest Neighbors	4
3.2	Support Vector Machine (SVM)	6
3.3	Naive Bayes	8
3.4	Logistic Regression	9
4	Deep learning approaches	11
4.1	Preprocessing	11
4.2	Recurrent Neural Networks (RNN)	12
4.3	Long short-term memory (LSTM)	14
5	Experimental results	16
5.1	Evaluation metrics	16
5.2	Full experimental results	17
5.3	Demo	19
6	Conclusion	19

1 Introduction

1.1 Project description

This project presents the development of an SMS spam classification system using both machine learning and deep learning techniques. The pipeline consists of three main stages: data ingestion, model development, and deployment preparation. First, data from multiple sources is collected, explored, and preprocessed into structured training, validation, and testing sets. These are then used to train and evaluate various models including KNN, SVM, Naive Bayes, Logistic Regression and LSTM.

The best-performing models are selected based on validation metrics, and the final models are saved for deployment. Additional efforts are made to ensure model interpretability, usability via interface creation, and proper project documentation. By combining classic and modern AI approaches, this project contributes to enhancing digital communication safety by detecting and filtering spam messages effectively.

1.2 Work distributions

Upon debating, we've divided our tasks and our distributed work for the project as follows:

- Pham Thi Ngoc Anh: SVM, Logistic Regression, Slides Presentation
- Tran Mai Duong: EDA, Data Preprocessing, Naive Bayes
- Le Quang Huy: LSTM, Demo
- Vu Thuy Linh: KNN, Material Research, Report

1.3 The Dataset and Exploratory Data Analysis

During exploratory data analysis, we examined the class distributions of the three datasets. The UCI SMS Spam Collection [1] is heavily imbalanced, with only 13.4% spam messages. The Telegram dataset [2] is more balanced, with 29.5% spam, while the Enron-Spam dataset [3] is nearly balanced, containing 50.9% spam messages, making it the most diverse in terms of spam density. After merging the three datasets, we obtained a combined corpus of approximately 59,000 SMS messages. The resulting dataset has a moderately imbalanced class distribution, with 59.9% ham and 40.1% spam message, providing a rich and diverse source for training robust classifiers. And we will split the dataset into 3 parts, 80% of the dataset for training, and 10% each for testing and validation set.

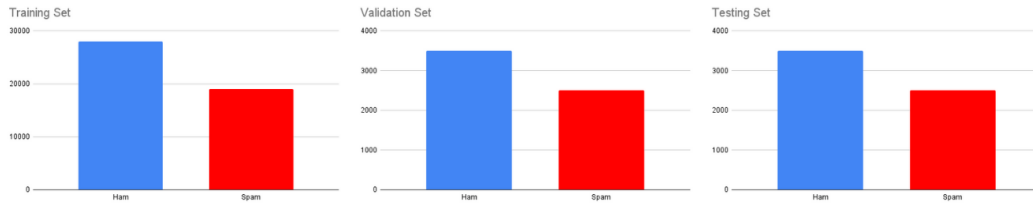


Figure 1: Partition of label in each set after splitting

2 Preprocessing Data

2.1 Cleaning and tokenizing texts

We will apply the basic techniques to make our text data clean:

- Remove “(; . * ? ,)” markup
- Remove punctuation marks
- Remove redundant whitespace
- Convert them into lowercase letters
- Remove stopwords: Some very common words like ”a”, ”an”, ”the”, ”me”, are very common and doesn't contain a lot of meaningful information for predicting the label of the message
- Tokenization: The process of replacing data with unique identification symbols so that we can assign ”meanings” to it through the process of learning.

Tomatoes are one of the most popular plants for vegetable gardens.
 Tip for success: If you select varieties that are resistant to disease and
 pest s, growing tomatoes can be quite easy. For experienced garden ers
 looking for a challenge, there are endless heirloom and specialty
 varieties to cultivate. Tomato plants come in a range of sizes.

Figure 2: Example of tokenization

- Stemming: Some words in English have different forms, for example, the word ”Eat”, it has a lot of variants like it's past tense: ”Ate”, or noun form: ”Eating”, ... Thus when we create a vocabulary to store them, it will take extra space to store different variants of word without providing many extra information, so the

solution is to stem those words down into simpler form. The stopwords vocabulary and word tokenized is imported from *nltk* library.

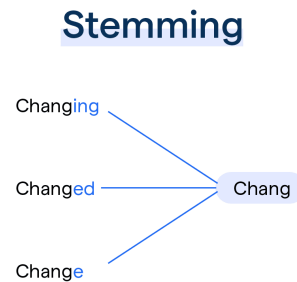


Figure 3: Example of words before and after stemming

2.2 Vectorizing texts

2.2.1 Bag of Words

The Bag of Words model represents text as an unordered collection or a "bag" of words, disregarding grammar and word order. It focuses solely on the presence and frequency of words in a document. Each document is represented as a numerical vector, where each dimension corresponds to a word from the vocabulary, and the value in each dimension represents the frequency of that word in the document.

For our project, the vocabulary of a 59000 sentence - dataset includes more than 30000 words, making our dataset very sparse, to deal with this problem, we choose to keep the most frequent 3000 and 1000 words.

2.2.2 Term frequency - Inverse document frequency

TF-IDF aims to evaluate the importance of a word in a document relative to its frequency in the document and the entire corpus. TF measures how frequently a term appears in a document. It is calculated as the number of times a word appears in a document divided by the total number of words in that document. IDF measures the rarity or importance of a word across the entire corpus. It's calculated as the logarithm of the ratio of the total number of documents to the number of documents containing the word. The overall TF-IDF score is obtained by multiplying the TF value and the IDF value for each term in a document. TF-IDF will assign higher values to words that are frequent in a document but rare across other documents.

$$TF(t, d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d}$$

$$IDF(t, D) = \log \left(\frac{\text{Total number of documents in the corpus } |D|}{\text{Number of documents containing term } t + 1} \right)$$

$$TF-IDF(t, d, D) = TF(t, d) * IDF(t, D)$$

The same problem applied to our TF-IDF vectorization, we will only keep the most frequent 3000 words and 1000 words as the vectorization option.

3 Machine Learning algorithms

3.1 K-Nearest Neighbors

K-nearest neighbor (KNN) [4] is a supervised learning algorithm in Machine Learning. It is used for classification and regression by picking k closest training examples in the dataset to predict the output (label) of new data. For classification problems, the class of the data is decided based on "major votes". This means that around K nearest data (in this example vectors) around them based on geometry or different metric distances, which class has the most appearance will be chosen to be the class of the testing data. This is the first algorithm that we will propose to design a system. Regarding picking k nearest neighbors, we will consider 2 main ways to choose those vectors.

$$Z = \frac{X - \mu}{\sigma}$$

Parameters

- **K (Number of neighbors):** The most important parameter for the algorithm, determine the suitable number of K will affect the overall performance of KNN, we determine the number of K using GridSearch with Cross-validation evaluation on a sample from the training dataset to reduce computational time.
- **Distance metrics:** Determining how "similar" those data points are also one of the crucial steps that can affect the performance of KNN. There are various distance measures that can be experimented, the two distance measure that we experimented in this project is:
 - **Euclidean distance:** It is the straight-line distance between two points in the data space. One of the most common distance measures used in the KNN model. It is especially useful when attributes in the dataset is continuous and the attributes scale are similar

$$d(\mathbf{p}, \mathbf{q}) = \sqrt{\sum_{i=1}^N (p_i - q_i)^2}$$

- **Manhattan distance:** The second way to compute is the Manhattan distance. We just need to calculate the sum of the deviations instead of computing the square root and then do the same as the first way. Suitable for cases where the distance along the axes (or dimensions) is more important than the direct Euclidean distance, such as in grid-based representations.

$$d(\mathbf{p}, \mathbf{q}) = \sum_{i=1}^N |p_i - q_i|$$

- **Weighting function:** For KNN, we are assuming every nearest neighbor's weight is equal, which means that every neighbor in that K-nearest neighbors set has the same weight, even the K-th closest one and the closest one to the testing vector. In practice it may have some bad effects, resulting in bad/inaccurate predictions. So we need to assign weight to them. It means that the nearest vector to the testing vector has more "weight" when we consider the label of the testing vector. The closer the vector is to the testing one, it will have more influence on the output of the data than the further ones. Any function can be used as a kernel function for the weighted KNN classifier whose value decreases as the distance increases.

$$y_{new} = \arg \max_{v \in V} \sum_{i=1}^K w_i * \delta(v, f(x_i))$$

where:

$$\delta(a, b) = \begin{cases} 1 & \text{if } a = b \\ 0 & \text{otherwise} \end{cases}$$

In this project, we will experiment using these weights

- **Uniform weight:** This weighting means that every neighbor in K-nearest neighbors have the same influence on the predicted label of the new instance. It means that in mathematical expression: $w_i = 1 \quad \forall x_i \in K$
- **Distance weight:** The weighting is based on the actual distance metric that we used in this project. The closer the distance the larger the weight it will have. The distance weight is given as follows:

$$w_i = \frac{1}{d_i^2}$$

- **Gaussian weight:**

$$w_i = e^{-\frac{d_i^2}{2\sigma^2}}$$

For this specific project, to pick the right parameters, We perform a grid search over

a range of hyperparameters to identify the optimal values, using the GridSearchCV class in Python’s scikit-learn library to automate this process. The parameter grid is: `{‘n_neighbors’: [2, 5, 10], ‘weight’ = [‘uniform’, ‘distance’, ‘weight’ (Note: ‘weight’ likely refers to Gaussian weight or some other specific kernel, clarify if possible. The OCR has ‘weight’ repeated. Assuming Gaussian based on context))]}`

After performing the search on each embedding type and different dimensions of data using 5-fold cross validation, We found out that `{‘weight’: ‘distance’, ‘n_neighbors’: 2}` to be the optimal parameters for 4 of embedding options. The result of the optimal configurations for each of embedding type is displayed at the table below:

Embed	Train F1	Train bal acc	Val F1	Val bal acc	Test F1	Test bal acc
KNN-TFIDF1000	0.8292	0.8547	0.8421	0.8069	0.8349	0.8581
KNN-TFIDF3000	0.8461	0.8670	0.7491	0.7989	0.7407	0.7934
KNN-BOW1000	0.8532	0.8762	0.8114	0.8414	0.8037	0.8366
KNN-BOW3000	0.8523	0.8740	0.7991	0.8321	0.7978	0.8319

Table 1: Experimental result of KNN

3.2 Support Vector Machine (SVM)

SVM is a linear classification method, whose main purpose is to define a separator in the search space that can separate different classes. This separator is commonly referred to as a hyperplane. One of the advantages of this SVM method is that it is quite good at classifying high-dimensional data because the method tries to determine the optimal direction of discrimination in the feature space by examining the right feature combination.

There can be challenges that appear when trying to categorize given data. The margins concluded by SVMs can turn out to be too tightly strict. In this case, a “soft margin” can be introduced. In clearer explanation, the hyperplane corresponding to this soft margin will accept some data points that fall into the false category, paving the path for a more effective classification with a higher confidence level.

The data presented to SVMs can sometimes be unable to be classified linearly. However, SVMs can try to handle the situation by applying kernel functions to transform the data to a space of higher dimensions, thus giving possibilities of finding a feasible hyperplane. While common kernel functions such as linear, polynomial, or radial basis function (RBF) kernel can be utilized, users can also specify their own function to aid in classifying the given data.

Parameters

- **C (Regularization parameters):** Directly affects how false classifications are allowed during the classification process. For large values of C, the optimization

will choose a smaller-margin hyperplane if that hyperplane does a better job of getting all the training points classified correctly. Conversely, a very small value of C will cause the optimizer to look for a larger margin separating the hyperplane, even if that hyperplane misclassifies more points.

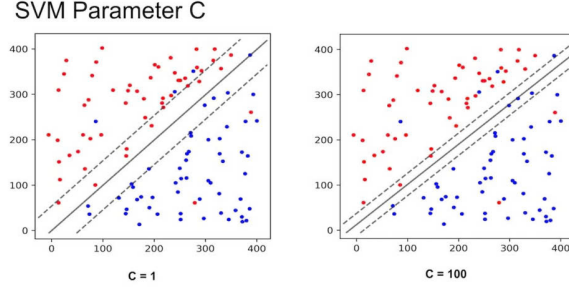


Figure 4: Example on the effect of constant C

- **Kernel functions:** Type of functions that we used to project our data to higher-dimensional space, so it can increase the chance of finding better hyperplanes for classification. Three types of kernel functions that we experimented with in this project are:

- **Linear kernel functions:** The linear kernel does not perform any transformation on the data, and it assumes that the data is already linearly separable or has a linear relationship

$$k(\mathbf{x}, \mathbf{z}) = \mathbf{x}^T \mathbf{z}$$

- **Polynomial kernel functions:** The polynomial kernel can model non-linear and polynomial relationships between the data points, and it can vary its complexity and shape by changing the r , γ and d constants

$$k(\mathbf{x}, \mathbf{z}) = (r + \gamma \mathbf{x}^T \mathbf{z})^d$$

- **Radial Basis Function (RBF):**

$$k(\mathbf{x}, \mathbf{z}) = \exp(-\gamma \|\mathbf{x} - \mathbf{z}\|^2)$$

- **Sigmoid kernel function:**

$$k(\mathbf{x}, \mathbf{z}) = \tanh(\gamma r + \mathbf{x}^T \mathbf{z})$$

Then we use GridSearch with different constant C and different kernel functions to find out the optimal hyperparameters for this specific task. The parameter grid is: $\{C\}$: [0.1,

1, 10, 100], 'kernel' = ['linear', 'poly', 'rbf', 'sigmoid']}. Through the process, we've found that {'C' = 10, 'kernel' = 'rbf'} is the most optimal configuration of this algorithm for this project.

Embed	Train F1	Train bal acc	Val F1	Val bal acc	Test F1	Test bal acc
SVM-TFIDF1000	0.9680	0.9711	0.9368	0.9441	0.9333	0.9402
SVM-TFIDF3000	0.9796	0.9807	0.9591	0.9528	0.9466	0.9533
SVM-BOW1000	0.8633	0.8811	0.8584	0.8770	0.8538	0.8739
SVM-BOW3000	0.8841	0.8975	0.8787	0.8932	0.8690	0.8856

Table 2: Experimental result of SVM

3.3 Naive Bayes

The Naive Bayes Algorithm is a popular classification technique based on Bayes' Theorem. It considers the features of the input data, with the assumption that these features appear independently of each other, to predict which class an item belongs to. Despite that this assumption generally does not hold in real-life situations, the Naive Bayes classifier is still widely used because of its high accuracy and low computational complexity.

The likelihood of features has an impact on Naive Bayes' probability calculation. Hence, there are three popular types of Naive Bayes: Gaussian, Multinomial, and Bernoulli. For our problem of SMS Spam classification, Multinomial Naive Bayes is the most suitable option where the data are represented as word vector counts. This allows us to account for multiple occurrences of words.

Intuitively, let $X = \{x_1, x_2, \dots, x_n\}$ Where $x_1, x_2, \dots, x_n$ are words in message X . Let H be a hypothesis that message X is non-spam message, S is a hypothesis that X is a spam message. According to Bayes's Theorem, we have:

$$P(H|X) = \frac{P(X|H)P(H)}{P(X)}, \quad P(S|X) = \frac{P(X|S)P(S)}{P(X)}$$

Where $P(H|X)$, $P(S|X)$ are posterior probabilities, $P(X|H)$, $P(X|S)$ are likelihoods, $P(H)$, $P(S)$ are class prior probabilities. To decide whether message X is spam or not, we compare $P(H|X)$ and $P(S|X)$. In short, we need to compare $P(X|H)P(H)$ and $P(X|S)P(S)$. And we calculate those via:

$$P(X|H)P(H) = P(x_1|H)P(x_2|H)\dots P(x_n|H)P(H)$$

$$P(X|S)P(S) = P(x_1|S)P(x_2|S)\dots P(x_n|S)P(S)$$

Where $P(H)$ and $P(S)$ are the proportion of non-spam, spam messages in the training dataset. And $P(x_i|H)$ and $P(x_i|S)$ can be determined by counting the number of occurrences

of x_i and divide it by the total number of words in the messages in the training set:

$$P(x_i|S) = \frac{\text{Number of occurrences of } x_i \text{ in labeled-spam messages}}{\text{Total number of words in labeled-spam messages}} = \frac{N_{s,x_i}}{N_s}$$

During this process, we may face the issue that the test email contains a word x_i not seen in the training data. The probability $P(x_i|S)$ and/or $P(x_i|H)$, when computed, would equal to 0. A workaround for this problem is to introduce a nonnegative smoothing prior α such that:

$$P(x_i|S) = \frac{N_{s,x_i} + \alpha}{N_s + n\alpha}$$

The optimal value of this parameter can be determined via the GridSearch process which we've conducted with 5-fold cross-validation evaluations. The grid search space of hyperparameters is: $\{\alpha: [0.1, 1, 10, 100]\}$. And with GridSearch, we've found the best parameters choice for each embedding type and dimensions, the results are shown in the table below:

Embed	Train F1	Train bal acc	Val F1	Val bal acc	Test F1	Test bal acc
NB-TFIDF1000	0.8727	0.8944	0.8714	0.8922	0.8751	0.8962
NB-TFIDF3000	0.8985	0.9162	0.8955	0.9125	0.9015	0.9191
NB-BOW1000	0.8566	0.8855	0.8565	0.8828	0.8584	0.8869
NB-BOW3000	0.8566	0.8855	0.8565	0.8828	0.8584	0.8869

Table 3: Experimental result of Naive Bayes

3.4 Logistic Regression

Logistic regression is a classification model that categorizes observations into one of two classes or multiple classes. In the context of spam filtering, it attempts to assign higher weights to text features that enhance its ability to distinguish between spam and non-spam.

Logistic regression solves this task by learning from the training set a vector of weights and a bias term. Each weight w_i represents how important that input feature is to the classification decision. Hence, we might expect the message which has meaning to have a high positive weight, and the gibberish text message to have a negative weight.

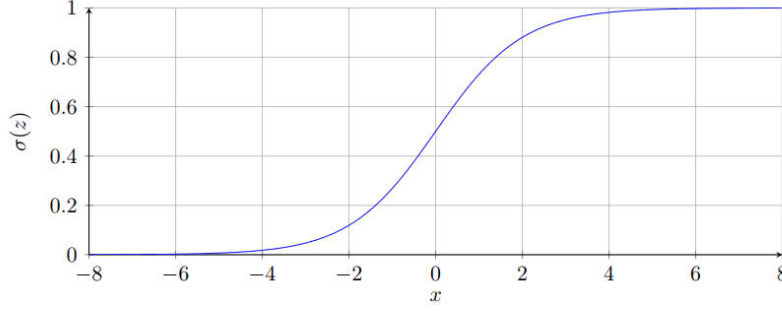


Figure 5: Sigmoid function $\sigma(z) = \frac{1}{1+e^{-z}}$

To make a decision, the classifier multiplies each feature x_i with its weight w_i , sums up the weighted features and adds the bias term b , results in a single number z .

$$z = \mathbf{w} \cdot \mathbf{x} + b$$

The logistic regression uses the logistic function to squeeze z between 0 and 1 to create a probability. The logistic function, shown graphically in Figure 5 is defined as:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

The possibility of $y = 1$ with the input $\mathbf{x}$ or the probability that the observation is spam can be expressed by

$$P(y = 1|\mathbf{x}) = \sigma(z)$$

We choose $\sigma(z) = 0.5$ as the decision threshold since the possibility that a message is spam or not is equal. Correspondingly, for $\sigma(z) > 0.5$, the observation is assigned to the 1 class (spam); otherwise, for $\sigma(z) \leq 0.5$, the observation is assigned to 0 class (ham). The closer the result of this function is to 1, the more likely the text is to be spam.

The cross-entropy loss function

The cross-entropy loss function is an objective function for learning and works well on classification task, especially binary classification. The model adjusts the parameters to make the predicted label $\hat{y}$ (where $\hat{y} = \sigma(z)$) as close as possible to the true label y (which is 0 or 1). In other words, it aims to minimize the loss function with loss is defined by

$$\text{Loss} = - \sum_{i=1}^{\text{output size}} y_i \log(\hat{y}_i)$$

We would like to learn weights that maximize the probability of the correct label $P(y|\mathbf{x})$. Since there are only two discrete outcomes (spam or non-spam), this follows a Bernoulli

distribution:

$$P(y|\mathbf{x}) = p^y \cdot (1 - p)^{1-y}$$

Using Maximum Likelihood Estimation since it helps find $\mathbf{w}$ and b so that the probability of predicting correct $P(y|x; w, b)$ is the highest, the cross-entropy loss function is:

$$L_{CE}(\hat{y}, y) = -[y \log(\sigma(\mathbf{w} \cdot \mathbf{x} + b)) + (1 - y) \log(1 - \sigma(\mathbf{w} \cdot \mathbf{x} + b))]$$

Parameters

- **C:** Inverse of regularization strength, smaller values specifies stronger regularization
- **Penalty (l1, l2):** specify the norm of the penalty. The level each technique improves the performance of our system depends on the characteristics of out dataset.

We use GridSearchCV to find the most suitable hyperparameters. The parameter grid is: 'penalty':['l1', 'l2'], 'solver':['liblinear'], 'C':[0.001, 0.01, 0.1, 1.0, 10.0, 100.0, 1000.0].

Embed	Train F1	Train bal acc	Val F1	Val bal acc	Test F1	Test bal acc
LR-BOW1000	0.9165	0.9272	0.9067	0.9197	0.9044	0.9178
LR-BOW3000	0.9564	0.9611	0.9304	0.9393	0.9289	0.9383
LR-TFIDF1000	0.9206	0.9324	0.9093	0.9230	0.9052	0.9198
LR-TFIDF3000	0.9522	0.9589	0.9262	0.9372	0.9266	0.9374

Table 4: Experimental result of Logistic Regression

4 Deep learning approaches

For the above algorithms, we only made use of the information of the all the words available that were in the sentences, and we did not make use of the position of those words. In practice, it is very important that we pay attention to the order of the words present in the sentence as well. For example the phrase “No like” means “dislike”, but our previous algorithm will not notice that pattern, and with deep learning, we can hopefully capture those dependencies. For this deep learning approaches, we will use different variants of the Recurrent neural networks (RNNs).

4.1 Preprocessing

The preprocessing process with deep learning approaches are pretty much similar to the one mentioned above. But now, we introduce an embedding layer to “map” the words in the vocabulary to a dense vector, this way deep learning can control how parameters

are learned, ensuring it will try to learn the best parameters possible for those embedding representation based on the training data.

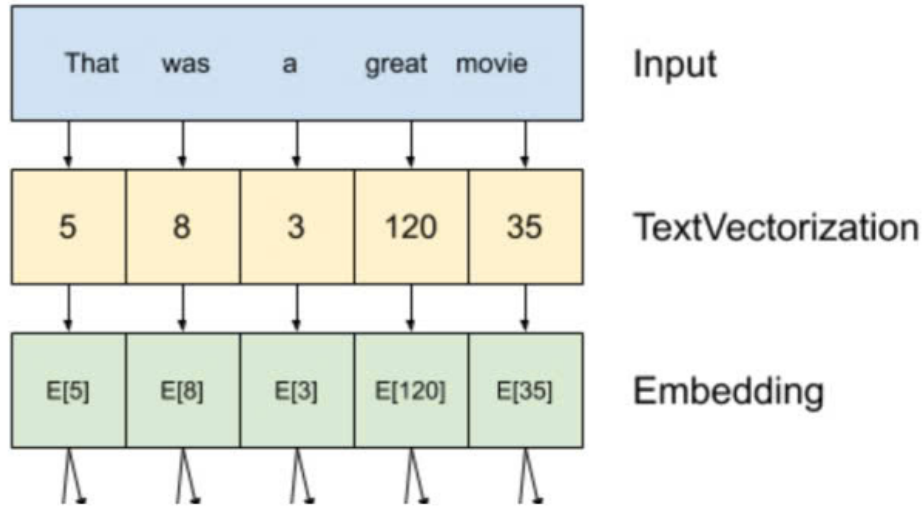


Figure 6: Example of embedding words

Denote the input batches of sentence $\mathbf{X} \in \mathbb{R}^{n \times l \times V}$, where n is the batch size, l is longest length of the sentence in that batch, V is the size of vocabulary and d as the embedding dimension of the embedding layer that we want to map our words in vocabulary too. Then the input of sentence at time step t (t -th word in that sentence) is $\mathbf{X}_t \in \mathbb{R}^{n \times V}$, where each vector in that time step is the one-hot representation of the word in that vocabulary. Then after the embedding layer, we obtain an embedded tensor for $\mathbf{X}_t$ is $\mathbf{E}_t \in \mathbb{R}^{n \times d}$. The tensor $\mathbf{E}_t$ is obtained as follows:

$$\mathbf{E}_t = \mathbf{X}_t \mathbf{W}_e$$

Where $\mathbf{W}_e \in \mathbb{R}^{V \times d}$ are learnable parameters weight matrix. And after this embedding layer, we can use those vector representations ($\mathbf{E}_t$) as input to our recurrent neural networks.

4.2 Recurrent Neural Networks (RNN)

For sequential data type, where data order really matters, and data points are dependent on other data points, original neural network (MLP) often fail to capture those relationships. So that's when RNNs come in. It is used to solve many problems relating to sequential data.

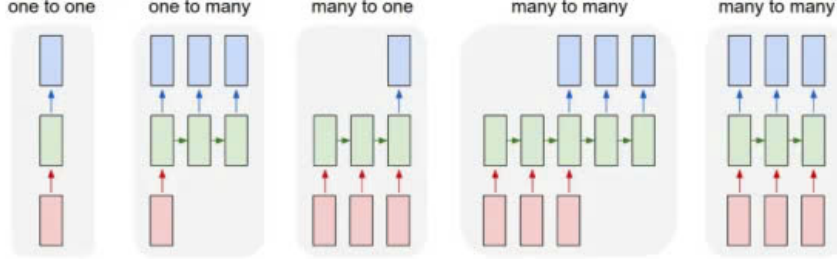


Figure 7: Some common problem types using RNN

Revisiting an fully connected network structure with a single hidden layer. Denote ϕ as the hidden layer's activation function, $\mathbf{X} \in \mathbb{R}^{n \times d}$, the hidden layer output $\mathbf{H} \in \mathbb{R}^{n \times h}$, and Output layer $\mathbf{O} \in \mathbb{R}^{n \times q}$ calculated as

$$\mathbf{H} = \phi(\mathbf{X}\mathbf{W}_{xh} + \mathbf{b}_h)$$

$$\mathbf{O} = \phi(\mathbf{H}\mathbf{W}_{hq} + \mathbf{b}_q)$$

Where $\mathbf{W}_{xh} \in \mathbb{R}^{d \times h}$, $\mathbf{b}_h \in \mathbb{R}^{1 \times h}$, number of hidden units as h , q as the number of hidden layer for the output layer. For other purposes like classification problem, we can map this output layer through a softmax function to retrieve a probability distribution.

Now let's denote $\mathbf{X}_t \in \mathbb{R}^d$ an input at timestep t , $\mathbf{H}_t \in \mathbb{R}^h$ the output at timestep t . Unlike MLP, We will save the hidden layer output $\mathbf{H}_{t-1}$ from previous time step and introduce a new parameter $\mathbf{W}_{hh} \in \mathbb{R}^{h \times h}$ to link the output of the hidden layer from previous timesteps to the current time step via this formula

$$\mathbf{H}_t = \phi(\mathbf{X}_t\mathbf{W}_{xh} + \mathbf{H}_{t-1}\mathbf{W}_{hh} + \mathbf{b}_h)$$

From the relationship between hidden layer outputs $\mathbf{H}_t$ and $\mathbf{H}_{t-1}$ of adjacent time steps, we know that these variables captured and retained the sequence's historical information up to their current time step, just like the state or memory of the neural network's current time step. And for timestep t , the output of the output layer is similar to the formula constructed by MLP above:

$$\mathbf{O}_t = \phi(\mathbf{H}_t\mathbf{W}_{hq} + \mathbf{b}_q)$$

Unlike MLP, where the number of learnable weight matrices are the number of hidden layers, RNNs only contains 3 weights matrices and 2 bias vectors, which are $\mathbf{W}_{xh} \in \mathbb{R}^{d \times h}$, $\mathbf{W}_{hh} \in \mathbb{R}^{h \times h}$, $\mathbf{b}_h \in \mathbb{R}^{1 \times h}$, $\mathbf{W}_{hq} \in \mathbb{R}^{h \times q}$ and $\mathbf{b}_q \in \mathbb{R}^{1 \times q}$. Where $\mathbf{b}_h, \mathbf{b}_q$ are the bias vectors of hidden layer and output layer, respectively.

In practice, RNNs suffer from vanishing or exploding gradients, which often prevents them from learning long-term dependencies in sequential data. Therefore, we do not experiment with traditional RNNs in this project but focus on improved versions instead.

4.3 Long short-term memory (LSTM)

To resolve the vanishing gradients problem, the architecture LSTM was introduced in 1997 [5], the LSTMs is also a resemblance of a standard recurrent neural networks structure. but here each recurrent node is replaced by a memory cell. Each memory cell contains an internal state, (a node with a self-connected recurrent edge) of fixed weight 1, ensuring that the gradient can pass across many time steps without vanishing or exploding.

Each memory cell is equipped with an internal state and a number of multiplicative gates that determine whether

- A given input should impact the internal state (which is the input gate)
- The internal state should be flushed to 0 (which is the forget gate)
- The internal state of a given neuron should be allowed to impact the cell's output (the output gate).

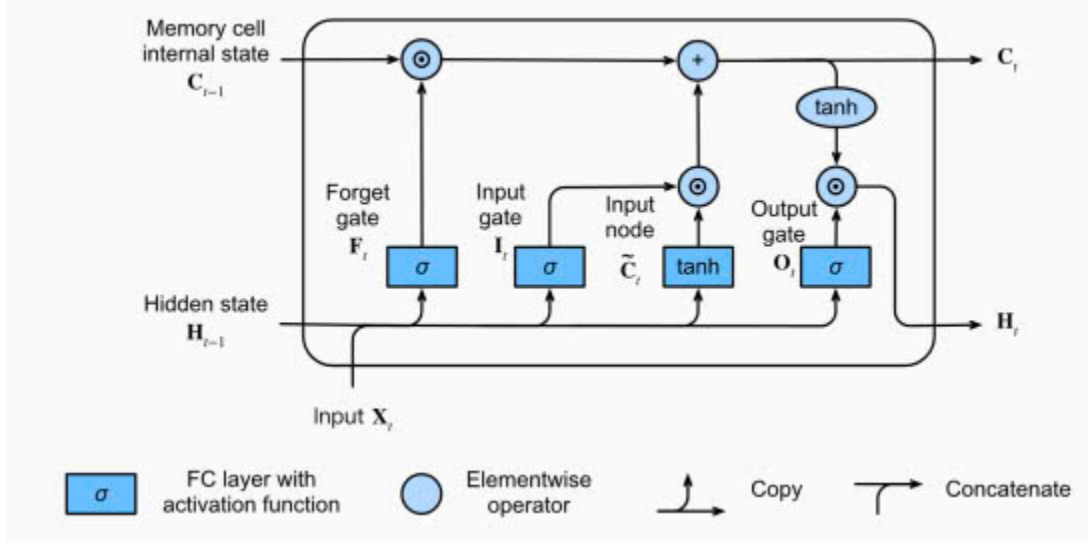


Figure 8: Overall structure of an LSTM Memory cell

4.3.1 Input Gate, Forget Gate, and Output Gate

The data feeding into the LSTM gates are the input at the current time step and the hidden state of the previous time step to compute the values of the input, forget, and output gates. As a result, values of the three gates are in the range of $(0, 1)$.

Mathematically, suppose that there are h hidden units (h -dimensional hidden unit), batch size is n , the dimensional input is d , the input at timestep t is $\mathbf{X}_t \in \mathbb{R}^{n \times d}$, hidden state of previous time step is $\mathbf{H}_{t-1} \in \mathbb{R}^{n \times h}$. Correspondingly, the gates at time step t are

defined as follows: the input gate is $\mathbf{I}_t \in \mathbb{R}^{n \times h}$, the forget gate is $\mathbf{F}_t \in \mathbb{R}^{n \times h}$, the output gate is $\mathbf{O}_t \in \mathbb{R}^{n \times h}$. They are calculated as follows:

$$\begin{aligned}\mathbf{I}_t &= \sigma(\mathbf{X}_t \mathbf{W}_{xi} + \mathbf{H}_{t-1} \mathbf{W}_{hi} + \mathbf{b}_i) \\ \mathbf{F}_t &= \sigma(\mathbf{X}_t \mathbf{W}_{xf} + \mathbf{H}_{t-1} \mathbf{W}_{hf} + \mathbf{b}_f) \\ \mathbf{O}_t &= \sigma(\mathbf{X}_t \mathbf{W}_{xo} + \mathbf{H}_{t-1} \mathbf{W}_{ho} + \mathbf{b}_o)\end{aligned}$$

With learnable parameters $\mathbf{W}_{xi}, \mathbf{W}_{xf}, \mathbf{W}_{xo} \in \mathbb{R}^{d \times h}$, $\mathbf{W}_{hi}, \mathbf{W}_{hf}, \mathbf{W}_{ho} \in \mathbb{R}^{h \times h}$, $\mathbf{b}_i, \mathbf{b}_f$ and $\mathbf{b}_o \in \mathbb{R}^{1 \times h}$.

4.3.2 Input node

Next we design the input node. Since we have not specified the action of the various gates yet, we first introduce the candidate memory cell $\tilde{\mathbf{C}}_t \in \mathbb{R}^{n \times h}$. Its computation is similar to that of the three gates described above, but using a tanh function with a value range $(-1, 1)$ as the activation function. This leads to the following equation at time step t :

$$\tilde{\mathbf{C}}_t = \tanh(\mathbf{X}_t \mathbf{W}_{xc} + \mathbf{H}_{t-1} \mathbf{W}_{hc} + \mathbf{b}_c)$$

With learnable parameters $\mathbf{W}_{xc} \in \mathbb{R}^{d \times h}$, $\mathbf{W}_{hc} \in \mathbb{R}^{h \times h}$, $\mathbf{b}_c \in \mathbb{R}^{1 \times h}$.

4.3.3 Memory cell internal state

In LSTMs, the input gate $\mathbf{I}_t$ governs how much we take new data into account via $\tilde{\mathbf{C}}_t$ and the forget gate $\mathbf{F}_t$ addresses how much of the old cell internal state $\mathbf{C}_{t-1} \in \mathbb{R}^{n \times h}$ we retain. Using the $\odot$ (elementwise) product operator we arrive at the following update equation:

$$\mathbf{C}_t = \tanh(\mathbf{F}_t \odot \mathbf{C}_{t-1} + \mathbf{I}_t \odot \tilde{\mathbf{C}}_t)$$

4.3.4 Hidden state

Last, we need to define how to compute the output of the memory cell $\mathbf{H}_t \in \mathbb{R}^{h \times h}$ as seen by other layers. This is where the output gate comes into play. In LSTMs, we first apply tanh activation functions to the memory cell internal state $\mathbf{C}_t$ and then apply another point-wise multiplication, this time with the output gate. This ensures that the values of $\mathbf{H}_t$ are always in the interval $(-1, 1)$.

$$\mathbf{H}_t = \mathbf{O}_t \odot \tanh \mathbf{C}_t$$

Whenever the output gate is close to 1, we allow the memory cell internal state to impact the subsequent layers uninhibited, whereas for output gate values close to 0, we prevent the current memory from impacting other layers of the network at the current time

step. Note that a memory cell can accrue information across many time steps without impacting the rest of the network (as long as the output gate takes values close to 0), and then suddenly impact the network at a subsequent time step as soon as the output gate flips from values close to 0 to values close to 1.

5 Experimental results

5.1 Evaluation metrics

In pattern recognition, information retrieval, object detection, and classification (machine learning), the confusion matrix is a performance metrics that allow us to have a better measure of evaluating our classification models. The confusion matrix is a 2x2 matrix, where:

- **True positives (TP):** The number of relevant (E.g: Positive) instances that were accurately predicted by our models.
- **True negatives (TN):** The number of irrelevant (E.g: Negative) instances that were accurately predicted by our models.
- **False positives (FP):** The number of instances that were classified as relevant (E.g: Spam) but are actually irrelevant (E.g: Non-spam)
- **False negatives (FN):** The number of instances that were classified as irrelevant (E.g: Non-spam) but are actually relevant (Spam)

		Actual Values	
		Positive (1)	Negative (0)
Predicted Values	Positive (1)	TP	FP
	Negative (0)	FN	TN

Figure 9: Confusion matrix

With those measurements, we will have our metrics calculated as follows:

- **Accuracy:** is the fraction of correctly predicted instances among all instances, written as:

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + TN + FN}$$

- **Precision** (positive predictive value) is the fraction of relevant instances among the retrieved instances. Written as a formula:

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Recall** (also known as sensitivity) is the fraction of relevant instances that were retrieved. Written as a formula:

$$\text{Recall} = \frac{TP}{TP + FN}$$

- **Balanced accuracy:** also called an arithmetic mean of true positive rate (Recall) and true negative rate

$$\text{Balanced accuracy} = \frac{1}{2} \left(\frac{TP}{TP + FN} + \frac{TN}{TN + FP} \right)$$

(Note: The OCR'd formula on page 24 had 'TP/(TP+FN) + TN/(TN+FP)' which would sum to >1 often. Standard is average, so '1/2 (...)' or divided by 2.)

- **F1-score:** A harmonic means of Precision and Recall, a high F1-score means that both Precision and Recall of the model are high as well

$$\text{F1-score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

For this project, we will use F1-score and balanced accuracy to measure our models performances.

5.2 Full experimental results

5.2.1 Machine learning approaches

From the table, we observe that SVM with TF-IDF3000 achieves the best overall performance across all sets, with the highest F1-score and balanced accuracy on both validation and test data. Logistic Regression also performs competitively, offering strong results and stable generalization, particularly with TF-IDF. While Naive Bayes and KNN are computationally simpler and faster to train, their accuracy is generally lower. Among all combinations, models using TF-IDF with 3000 features tend to outperform those using Bag-of-Words or lower-dimensional embeddings.

Embed	Train F1	Train bal acc	Val F1	Val bal acc	Test F1	Test bal acc
KNN-TFIDF1000	0.8292	0.8547	0.8421	0.8069	0.8349	0.8581
KNN-TFIDF3000	0.8461	0.8670	0.7491	0.7989	0.7407	0.7934
KNN-BOW1000	0.8523	0.8762	0.8114	0.8141	0.8037	0.8366
KNN-BOW3000	0.8523	0.8740	0.7991	0.8321	0.7978	0.8319
SVM-TFIDF1000	0.9680	0.9711	0.9368	0.9411	0.9333	0.9402
SVM-TFIDF3000	0.9796	0.9807	0.9591	0.9528	0.9466	0.9533
SVM-BOW1000	0.8633	0.8811	0.8584	0.8770	0.8538	0.8739
SVM-BOW3000	0.8841	0.8975	0.8787	0.8932	0.8690	0.8856
NB-TFIDF1000	0.8727	0.8944	0.8714	0.8922	0.8751	0.8962
NB-TFIDF3000	0.8985	0.9162	0.8955	0.9125	0.9015	0.9191
NB-BOW1000	0.8352	0.8660	0.8334	0.8616	0.8375	0.8679
NB-BOW3000	0.8566	0.8855	0.8565	0.8828	0.8584	0.8869
LR-BOW1000	0.9165	0.9272	0.9067	0.9197	0.9044	0.9178
LR-BOW3000	0.9564	0.9611	0.9304	0.9393	0.9289	0.9383
LR-TFIDF1000	0.9206	0.9324	0.9093	0.9230	0.9052	0.9198
LR-TFIDF3000	0.9522	0.9589	0.9262	0.9372	0.9266	0.9374

Table 5: Full Experimental Results of Machine Learning Models

5.2.2 Deep learning approaches

Configs	Train F1	Train bal acc	Val F1	Val bal acc	Test F1	Test bal acc
LSTM200	0.9598	0.9618	0.9363	0.9386	0.9366	0.9412

Table 6: Experimental result of LSTM

We can see that the 1-layer LSTM model demonstrated strong generalization and competitive performance. While deeper architectures may yield further improvements, we limited the design due to computational constraints.

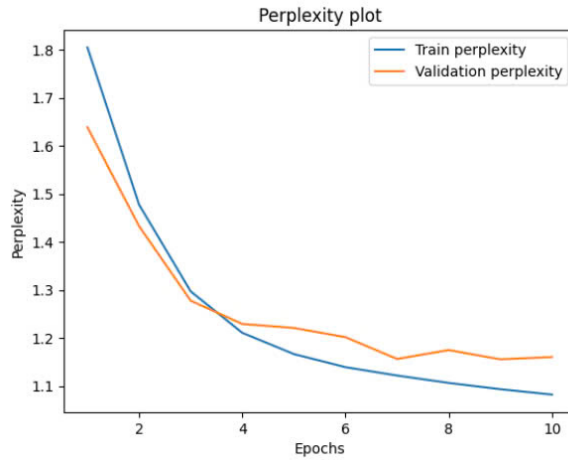


Figure 10: Perplexity values through training of LSTM

5.3 Demo

We built a lightweight GUI using ‘tkinter’ library that lets users input a message, select a model, and get real-time spam/ham predictions. Each model loads its corresponding vectorizer and pre-trained weights to process the input upon clicking “Process”.

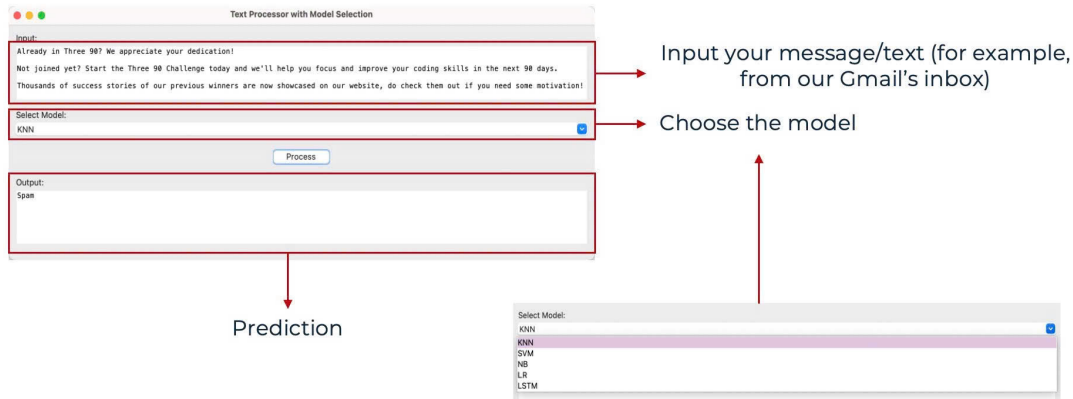


Figure 11: Demo GUI for SMS Spam classification

The image above illustrates a demo example where the user inputs a promotional message and selects KNN as the classification model. After processing, the system returns “Spam” as the prediction. This demonstrates how the trained models can be interactively used for practical message filtering via a simple GUI.

6 Conclusion

Working on this project has provided us with valuable practical experience in applying machine learning to a real-world classification task. Throughout the process, we encountered several challenges—from handling imbalanced datasets to tuning hyperparameters and selecting suitable vectorization methods—that required both theoretical understanding and hands-on experimentation.

Among the models evaluated, Support Vector Machine (SVM) consistently delivered the best performance, particularly when paired with TF-IDF features. Logistic Regression also proved to be a strong and reliable baseline, while Naive Bayes and KNN showed faster training times but slightly lower accuracy. The deep learning model using LSTM performed competitively, confirming its ability to capture sequential patterns even with a simple architecture.

Thank you, Assoc. Prof. Nguyen Hong Thanh and PhD. Dinh Viet Sang for reading until the end of our group’s Capstone project report. If there are any mistakes, or misconceptions in the report, we would like to hear the feedback, and comments from you on our group’s report.

References

- [1] *SMS Spam Collection* from UC Irvine. Machine Learning Repository, 2012.
Citation: <https://archive.ics.uci.edu/dataset/228/sms+spam+collection>
- [2] *Telegram Spam or Ham* from Kaggle, last update in 2023.
Citation: <https://www.kaggle.com/datasets/mexwell/telegram-spam-or-ham>
- [3] *Enron Spam Dataset* from Hugging Face, 2005.
Citation: https://huggingface.co/datasets/SetFit/enron_spam
- [4] A. Mucherino, P. J. Papajorgji, and P. M. Pardalos. k-Nearest Neighbor Classification, pages 83-106. Springer New York, New York, NY, 2009.
- [5] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural computation*, 9:1735-80, 12 1997.